

How we Implement sonar using API's in web flow is briefly defined at this document.

Below is the link of Sonar GraphQL Documentation we use as a reference:

<https://api.sonar.software/>

We have created 5 API's for sonar using express js.

In env file there are 2 required constants:

AUTH_TOKEN and BASE_URL

1: /create-serviceable-address

Using this API's we are creating a serviceable address in sonar it is required for create active accounts in sonar. Serviceable should be unique and it will be used only in 1 account.

Serviceable Address Api will return an ID. Required inputs are address, address2 city zip latitude and longitude. Express JS code for API:

```
router.get('/create-serviceable-address', async function (req, res) {  
  var data = JSON.stringify({  
    query: `mutation {  
      createServiceableAddress(  
        input: {  
          line1 : "${req.query?.address}",  
          line2 : "${req.query?.address2}",  
          city : "${req.query?.city}",  
          subdivision: US_UM,  
          zip: "${req.query?.zip}",  
          country: US,  
          latitude: "${req.query?.latitude}",  
          longitude: "${req.query?.longitude}",  
        }  
      ){  
        id  
      }  
    }`,  
    variables: {}  
  });  
  
  var configAccount = {  
    method: 'post',  
    url: `${process.env.BASE_URL}`,  
    headers: {  
      'Authorization': `Bearer ${process.env.AUTH_TOKEN}`,  
      'Content-Type': 'application/json'  
    },  
    data : data  
  };  
  
  await axios(configAccount).then(resp => {  
    return res.json(resp.data);  
  }).catch(err => {  
    return res.json(err);  
  });  
});
```

2: /create-address

Using this API's we are creating an account in sonar. Account Api will return an ID. Required inputs are name serviceable address (return from serviceable address api) and email. Express JS code for API:

```
router.get('/create-account', async function (req, res) {  
  var data = JSON.stringify({  
    query: `mutation {  
      createAccount(  
        input: {  
          company_id: 1,  
          account_type_id:2,  
          account_status_id: 4,  
          name: "${req.query?.name}",  
          serviceable_address_id: "${req.query?.serviceable_address_id}",  
          primary_contact:{  
            name: "${req.query?.name}",  
            email_address: "${req.query?.email}",  
          }  
        }  
      )}{  
        id  
      }  
    }`,  
    variables: {}  
  });  
  
  var configAccount = {  
    method: 'post',  
    url: `${process.env.BASE_URL}`,  
    headers: {  
      'Authorization': `Bearer ${process.env.AUTH_TOKEN}`,  
      'Content-Type': 'application/json'  
    },  
    data : data  
  };  
  
  await axios(configAccount).then(resp => {  
    return res.json(resp.data);  
  }).catch(err => {  
    return res.json(err);  
  });  
});
```

3: /add-service-to-account

Using this API's we are adding selected service into the created account in sonar. This Api will add selected service for an account (service already created into sonar as well) Add Service to Account API will also return an ID. Required inputs are account_id, service_id and price.

Express JS code for API:

```
router.get('/add-service-to-account', async function (req, res) {  
  
  var data = JSON.stringify({  
    query: `mutation {  
      addServiceToAccount(  
        input: {  
          account_id: ${req.query?.account_id},  
          service_id: ${req.query?.service_id},  
          quantity: 1,  
          price_override: ${req.query?.price},  
          price_override_reason: "Multistep Form Culture Wireless",  
        }  
      ){  
        id  
      }  
    }`,  
    variables: {}  
  });  
  
  var configTicketComment = {  
    method: 'post',  
    url: `${process.env.BASE_URL}`,  
    headers: {  
      'Authorization': `Bearer ${process.env.AUTH_TOKEN}`,  
      'Content-Type': 'application/json'  
    },  
    data : data  
  };  
  
  axios(configTicketComment).then(function (resp) {  
    return res.json(resp.data);  
  })  
  .catch(function (error) {  
    return res.json(error);  
  });  
});
```

4: /create-ticket

Using this API's we have created a ticket into the created account in sonar. This Api will create a ticket user and description user will see all details requested from the website. This api will also return an ID. Required inputs are account_id, subject, description, due_date. Express JS code for API:

```
router.get('/create-ticket', async function (req, res) {  
  
  var data = JSON.stringify({  
    query: `mutation {  
      createInternalTicket(  
        input: {  
          subject: "${req.query?.subject}",  
          description: "${req.query?.description}",  
          status: OPEN,  
          priority: HIGH,  
          due_date: "${req.query?.due_date}",  
          ticketable_type: Account,  
          ticketable_id: ${req.query?.account_id},  
          ticket_group_id: 5  
        }  
      ) {  
        id  
      }  
    }`,  
    variables: {}  
  });  
  
  var configTicket = {  
    method: 'post',  
    url: `${process.env.BASE_URL}`,  
    headers: {  
      'Authorization': `Bearer ${process.env.AUTH_TOKEN}`,  
      'Content-Type': 'application/json'  
    },  
    data : data  
  };  
  
  await axios(configTicket).then(function (resp) {  
    return res.json(resp.data);  
  })  
  .catch(function (error) {  
    return res.json(error);  
  });  
  
});
```

5: /create-ticket

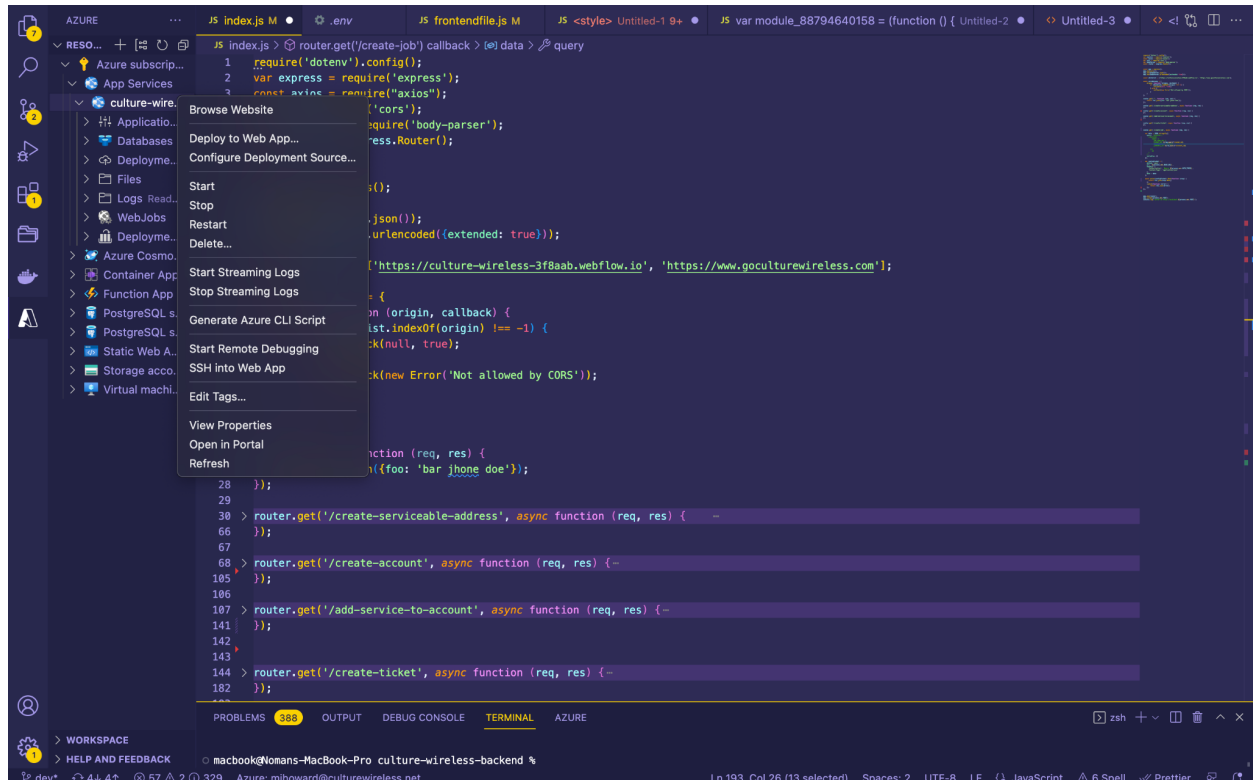
Using this API's we have created a job into the created account in sonar. This Api will create a job using ticket id in account. This api will also return an ID. Required inputs are account_id, ticket_id. Express JS code for API:

```
router.get('/create-job', async function (req, res) {  
  
  var data = JSON.stringify({  
    query: `mutation {  
      createJob(  
        input: {  
          job_type_id: 1,  
          ticket_id: ${req.query?.ticket_id},  
          jobbable_type: Account,  
          jobbable_id: ${req.query?.account_id},  
        }  
      ){  
        id  
      }  
    }`,  
    variables: {}  
  });  
  
  var configTicket = {  
    method: 'post',  
    url: `${process.env.BASE_URL}`,  
    headers: {  
      'Authorization': `Bearer ${process.env.AUTH_TOKEN}`,  
      'Content-Type': 'application/json'  
    },  
    data : data  
  };  
  
  await axios(configTicket).then(function (resp) {  
    return res.json(resp.data);  
  })  
  .catch(function (error) {  
    return res.json(error);  
  });  
});
```

We have deployed these APIs in azure account using app services, the below link will help us to install extension for azure and deploy APIs into azure account:

<https://learn.microsoft.com/en-us/azure/api-management/vscode-create-service-instance#prerequisites>

Below is example screenshot of our project:



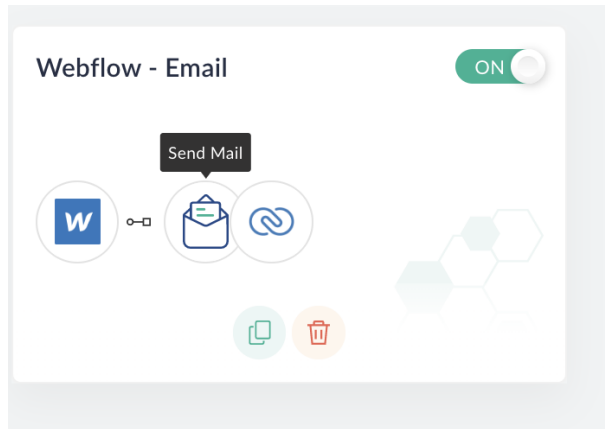
ZOHO implementation:

Q: Where is the custom module in zoho?

A: In zoho right panel go to CRM then from top menu 3 dots sub menu select customer or you can direct [Click Here](#):

Q: How do we connected zoho with webflow form?

A: In zoho right panel go to select Flow and then select webflow - Email like:

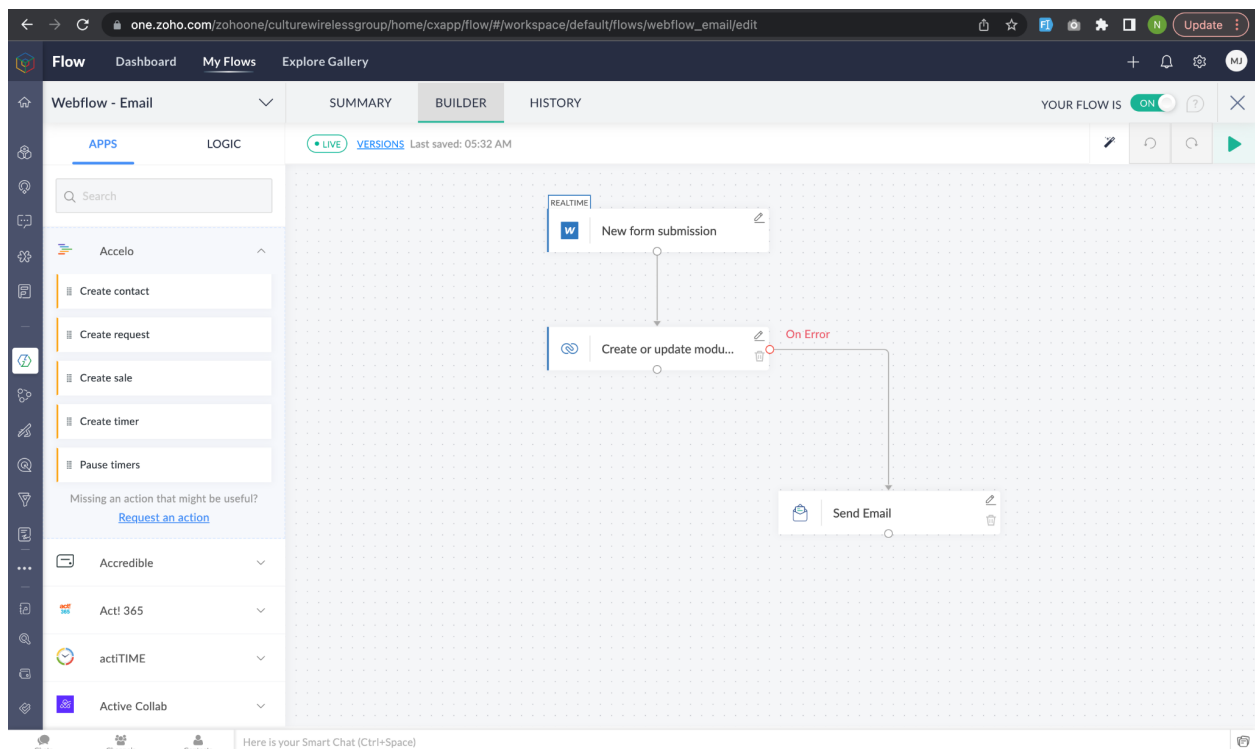


In this flow go to builder (2nd tab in top bar):

Here we have first connected webflow with zoho to get access to form and submitted data into zoho.

Then we attached a new hook with the help of right side page search zoho CRM and then select create and update module entry, then connected customer module entry with webflow form using given parameter by ZOHO.

OR check this article : <https://www.zoho.com/flow/apps/webflow/integrations/zoho-crm/>



Edit New Form Submission option:

The screenshot shows the Zoho Flow interface for editing a 'Form submitted' trigger. The left sidebar contains a list of actions under 'Webflow - Email', including 'Create contact', 'Create request', 'Create sale', 'Create timer', and 'Pause timers'. The main panel is titled 'Form submitted' and includes a description: 'Triggers when there is a new form submission'. The configuration options are as follows:

- Connection:** Webflow website (with a 'NEW' button and links for 'Test' and 'Reconnect').
- Variable Name:** trigger
- Site:** Culture Wireless
- Filter criteria:** Configure the conditions that trigger this flow. The criteria are: Name (dropdown), equals(case-insensitive) (dropdown), and Multy Step Form (checkbox). There is a 'Clear' button and an 'Add Group' link.

At the bottom right, there are 'BACK' and 'DONE' buttons.

Edit Create and update module entry option:

The screenshot shows the Zoho CRM interface for editing a 'Create or update module entry' action. The left sidebar contains a list of actions under 'Webflow - Email', including 'Create contact', 'Create request', 'Create sale', 'Create timer', and 'Pause timers'. The main panel is titled 'Zoho CRM - Create or update module entry' and includes a description: 'Creates a new module entry. Updates the module entry if it already exists'. The configuration options are as follows:

- Connection:** Zoho CRM (with a 'NEW' button and links for 'Test' and 'Reconnect').
- Variable Name:** culture_wireless_new_customer_request_1
- Module:** Customers
- Layout:** Standard
- Customer Name:** \${trigger.data['First Name']} \${trigger.data['Last Name']}
- Customer Owner:** (empty field)
- Email:** (empty field)

On the right side, there is an 'INSERT VARIABLE' section with a search bar and a list of variables:

- System Variables:** Variables holding predefined contextual information. Examples include 'Current date (e.g. 2020-12-22)' and 'Current datetime (e.g. 2021-05-30T23:30:30-05-...)'. There is a 'NEW' button.
- New form submission:** Webflow - Form submitted (with a refresh icon and a dropdown arrow).

At the bottom right, there are 'CANCEL' and 'DONE' buttons.

How to implement sonar api's in webflow?

First we have get access to form field to get data using this below code:

```
let username = $('#User-name').val()
let email = $('#Email').val()
let firstName = $('#First-Name-2').val()
let lastName = $('#Last-name').val()
let address = $('#Address').val()
let address2 = localStorage.getItem("ginputdata");
let latlong = localStorage.getItem("latlong").split(" ");
let latitude = latlong[0];
let longitude = latlong[1];
let city = $('#City').val()
let zip = $('#Zip-code').val()
let phone = $('input[name="Phone-number"]').val()
let acpCustomer = $('input[name="ACP-Customer"]:checked').val()
let subTotal = $('.totalprice')[0].innerText.replace('\n','')
let gst = $('.gst')[0].innerText.replace('\n','')
let qst = $('.qst')[0].innerText.replace('\n','')
let dueDate = $('#calender-date').val()
let timeSlot = $('#time-slot').val()
let sumDownloadSpeed1 = $('#sumDownloadSpeed1').text()
let totalmonthlyprice = $('.totalmonthlyprice')[0].innerText.replace('\n','')
var amountInteger = Number(totalmonthlyprice.replace(/^[^0-9.-]+/g,"")) * 100;
let selectedServiceId = null;
switch(sumDownloadSpeed1){
  case '100 Mbps':
    selectedServiceId = 5
    break;
  case '300 Mbps':
    selectedServiceId = 6
    break;
  case '500 Mbps':
    selectedServiceId = 7
    break;
  case '1 Gig':
    selectedServiceId = 8
    break;
  default:
    selectedServiceId = null;
    break;
}
```

After that we had implemented sonar api using this below code:

```
$.get('https://culture-wireless.azurewebsites.net/create-serviceable-address', serviceAddressParam, function (data, textStatus, jqXHR) {
    if(data.data.createServiceableAddress && textStatus == 'success'){
        console.log('Service created =====> ', data);

        let accountReqParam = {name: `${firstName} ${lastName}`, username: `${username}`, email: `${email}`, serviceable_address_id:
            data.data.createServiceableAddress?.id};
        $.get('https://culture-wireless.azurewebsites.net/create-account', accountReqParam, function (data1, textStatus, jqXHR) {
            console.log('Account created =====> ', data1);

            let accServiceReqParam = {account_id: data1.data.createAccount?.id, service_id: selectedServiceId, price: amountInteger }
            $.get('https://culture-wireless.azurewebsites.net/add-service-to-account', accServiceReqParam, function (data2, textStatus, jqXHR) {

                let fullDetails = `Installation Request by ${firstName} ${lastName} <br/> ACP User = ${acpCustomer} <br/> Selected Service = ${sumDownloadSpeed1}
                <br/> Subtotal = ${subTotal} <br/> gst = ${gst} <br/> qst = ${qst} <br/> totalmonthlyprice = ${totalmonthlyprice} <br/> Installation date = ${dueDate}
                <br/> Installation Time. = ${timeSlot}`;
                let accTicketReqParam = {subject: "Installation Request", description: fullDetails, due_date: `${dueDate}`, account_id: data1.data.createAccount?.id};
                $.get('https://culture-wireless.azurewebsites.net/create-ticket', accTicketReqParam, function (ticketData, textStatus, jqXHR) {
                    console.log('Ticket created =====> ', ticketData);
                    $(".gif-img").hide();
                    if(ticketData.data.createInternalTicket){
                        let accJobReqParam = {ticket_id: ticketData.data.createInternalTicket?.id, account_id: data1.data.createAccount?.id}
                        $.get('https://culture-wireless.azurewebsites.net/create-job', accJobReqParam, function (jobData, textStatus, jqXHR) {
                            console.log('Ticket created =====> ', jobData);
                            $(".gif-img").hide();
                            if(jobData.data.createJob){
                                $(".sonar-wait").hide();
                                $(".success-message-3.w-form-done").removeClass("hide-thanks");
                            }else{
                                $(".sonar-wait").text("Error in ticket create please logged in to sonar account and create ticket manually.");
                            }
                        });
                    }else{
                        $(".sonar-wait").text("Error in ticket create please logged in to sonar account and create ticket manually.");
                    }
                });
            });
        });
    }else{
        $(".gif-img").hide();
        $(".sonar-wait").text("Your order has been received. Check your email for confirmation. If there are any questions, a customer representative will contact you.");
    }
});
```